

CloudMLOps

AI Document Summarization Platform

Chimmili Nityanandh

GitHub: nitya200

chimmilinityanandh@gmail.com

Computer Science / Information Systems

Academic year: 2025-2026

The screenshot displays the CloudMLOps web application interface. On the left, a dark blue sidebar contains the CloudMLOps logo (a blue circle with 'CM' inside) and the text 'CloudMLOps DOCUMENT AI'. Below the logo, the heading 'Turn long documents into short answers.' is followed by a paragraph: 'An AI summarization platform built on a three-tier architecture: a React interface, a FastAPI business tier running FLAN-T5, and a PostgreSQL data tier that keeps every summary searchable.' A four-step process is listed in numbered boxes: 1. Upload a PDF, DOCX or TXT file, or paste raw text; 2. FastAPI extracts the text and picks a summary strategy; 3. FLAN-T5 generates an abstractive summary; 4. Everything is stored in PostgreSQL for search and rating. At the bottom of the sidebar, a row of technology logos is shown: React, FastAPI, PostgreSQL, FLAN-T5, Docker, GitHub Actions, and AWS App Runner. On the right, a light blue login form is visible. It starts with 'WELCOME BACK' and 'Sign in to your workspace'. There are input fields for 'Email address' (containing 'you@university.edu') and 'Password' (masked with dots). A blue 'Sign in' button is below these fields. At the bottom of the login form, it says 'No account yet? Create one'.

Project Overview

What the platform delivers

Three-tier web application for PDF, DOCX, and TXT summarization (or pasted text).

Solves slow manual reading with stored, searchable summaries and user ratings.

Built as a Cloud MLOps course project: architecture, AI, database, and AWS deployment.

Target users: students, analysts, and administrators monitoring quality.

Problem Statement

Why this system is needed

Document volume grows faster than time available for full reading.

Ad hoc notes are inconsistent and not searchable across a team.

Heavy AI models conflict with free-tier and HTTP timeout limits on cloud hosts.

Need: secure multi-user access, extraction pipeline, history, feedback, and DevOps.

Objectives

Primary implementation goals

Three-tier design

React, FastAPI, PostgreSQL

Document ingestion

PDF / DOCX / TXT + validation

AI summarization

FLAN-T5 local; extractive on AWS

User capabilities

Auth, history, download, ratings

Admin & MLOps

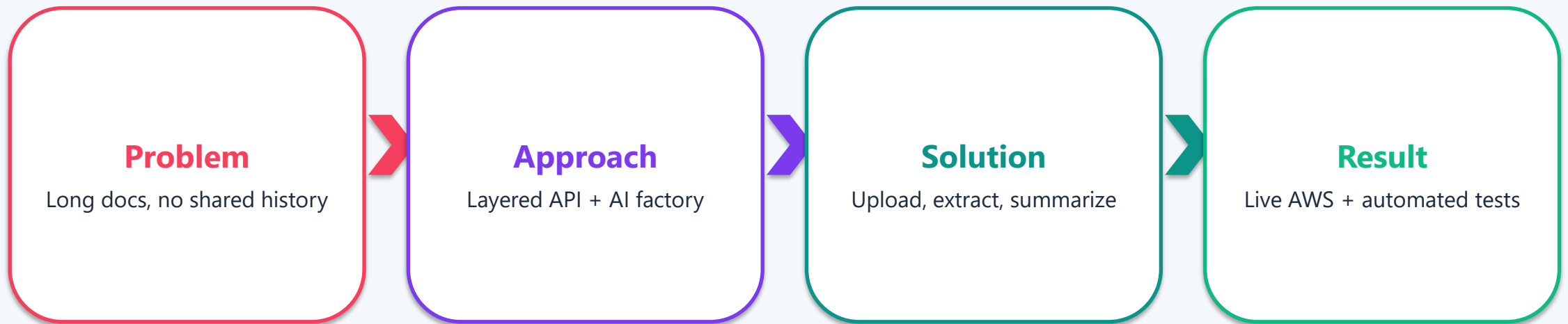
Metrics, users, model lifecycle

Cloud delivery

Docker, GitHub Actions, App Runner

Proposed Solution

Problem to production flow



Technologies Used

Stack from the repository

Frontend

React 19, Vite, React Router, Axios

Backend

Python 3.12, FastAPI, SQLAlchemy 2

Database

PostgreSQL 16, Alembic

AI / ML

Transformers, flan-t5-small, extractive

Extraction

PyMuPDF, python-docx

Cloud / DevOps

Docker, Actions, ECR, App Runner, Terraform

System Architecture

AWS production path

CI/CD

GitHub
ECR
Deploy

Browser / User

React frontend - AWS App Runner

FastAPI backend - AWS App Runner

PostgreSQL (RDS) + S3 uploads

CloudWatch + Secrets Manager

Key Features

Major modules

Authentication

JWT sessions, admin roles

Summarization

Text + documents, 3 lengths

History

Search and pagination

Feedback

Star ratings for quality

Admin dashboard

Usage and user management

Model lifecycle

UC-12 / UC-16 ROUGE pipeline

Live Deployment URLs

All verified working endpoints for this project

AWS production UI

<https://asqmhswfs.us-east-2.awsapprunner.com>

AWS production API

<https://p3jivcdmbf.us-east-2.awsapprunner.com>

Health check

<https://p3jivcdmbf.us-east-2.awsapprunner.com/health>

Swagger API docs

<https://p3jivcdmbf.us-east-2.awsapprunner.com/docs>

ReDoc API docs

<https://p3jivcdmbf.us-east-2.awsapprunner.com/redoc>

Reference demo UI (Netlify)

<https://cloudmlops.netlify.app>

Reference demo API (Render)

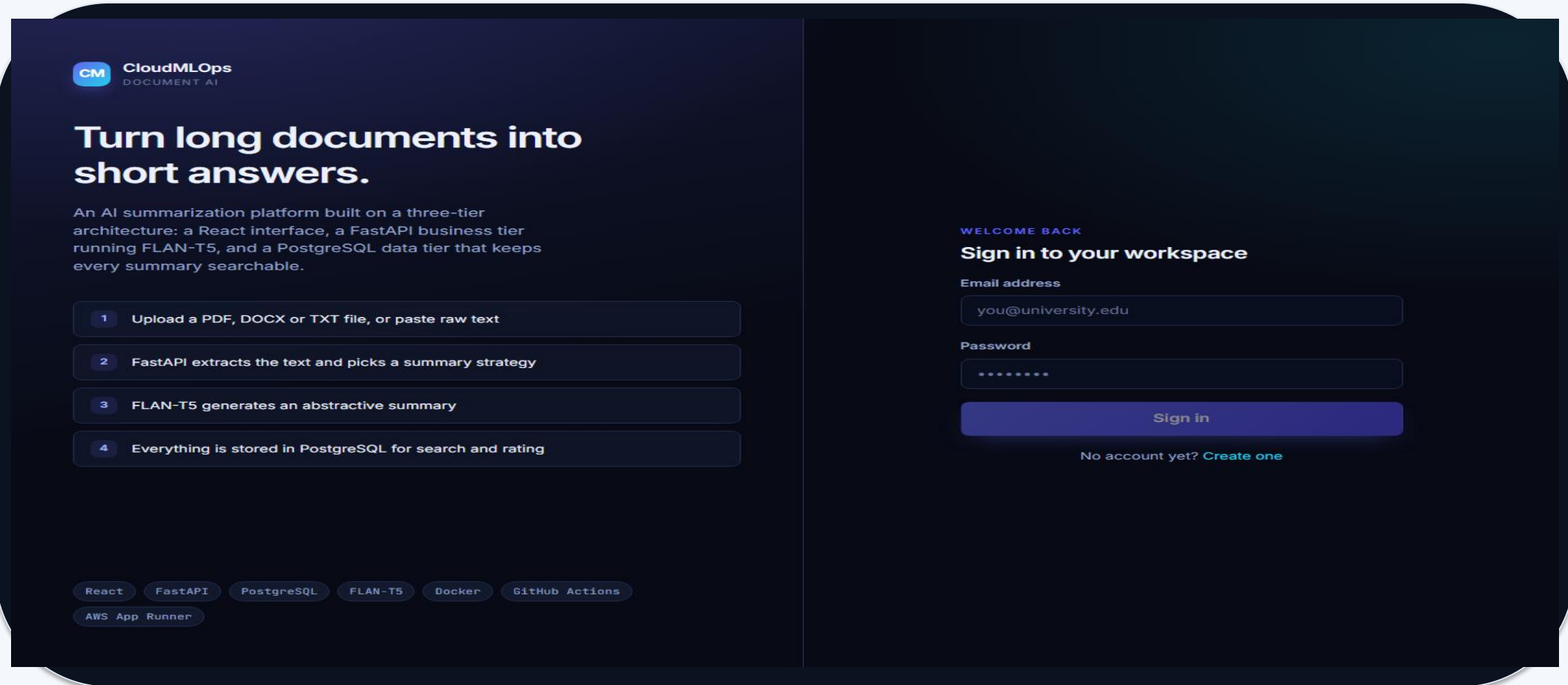
<https://cloudmlops.onrender.com>

GitHub repository

<https://github.com/nitya200/CloudMLOps>

Production Application

AWS App Runner frontend



Live CloudMLOps UI on AWS (home / login route).

API Documentation

Interactive OpenAPI on production API

CloudMLOps Document Summarizer 1.0.0 OAS 3.1

[/openapi.json](#)

CloudMLOps is a three-tier AI document summarization platform.

- **Presentation tier** - React single page application
- **Business tier** - this FastAPI service (authentication, extraction, summarization)
- **Data tier** - PostgreSQL

Upload a PDF, DOCX or TXT file (or paste raw text) and the FLAN-T5 model produces an abstractive summary. Every summary is stored so it can be searched, re-read, downloaded and rated.

Authentication

Call **POST** [/api/auth/login](#), then send the returned token as **Authorization: Bearer <token>** on every other request.

[CloudMLOps - Website](#)

MIT

Authorize



System Health and readiness probes.

GET [/health](#) Service health check

Authentication Registration, login, logout and profile.

POST [/api/auth/register](#) Register a new account

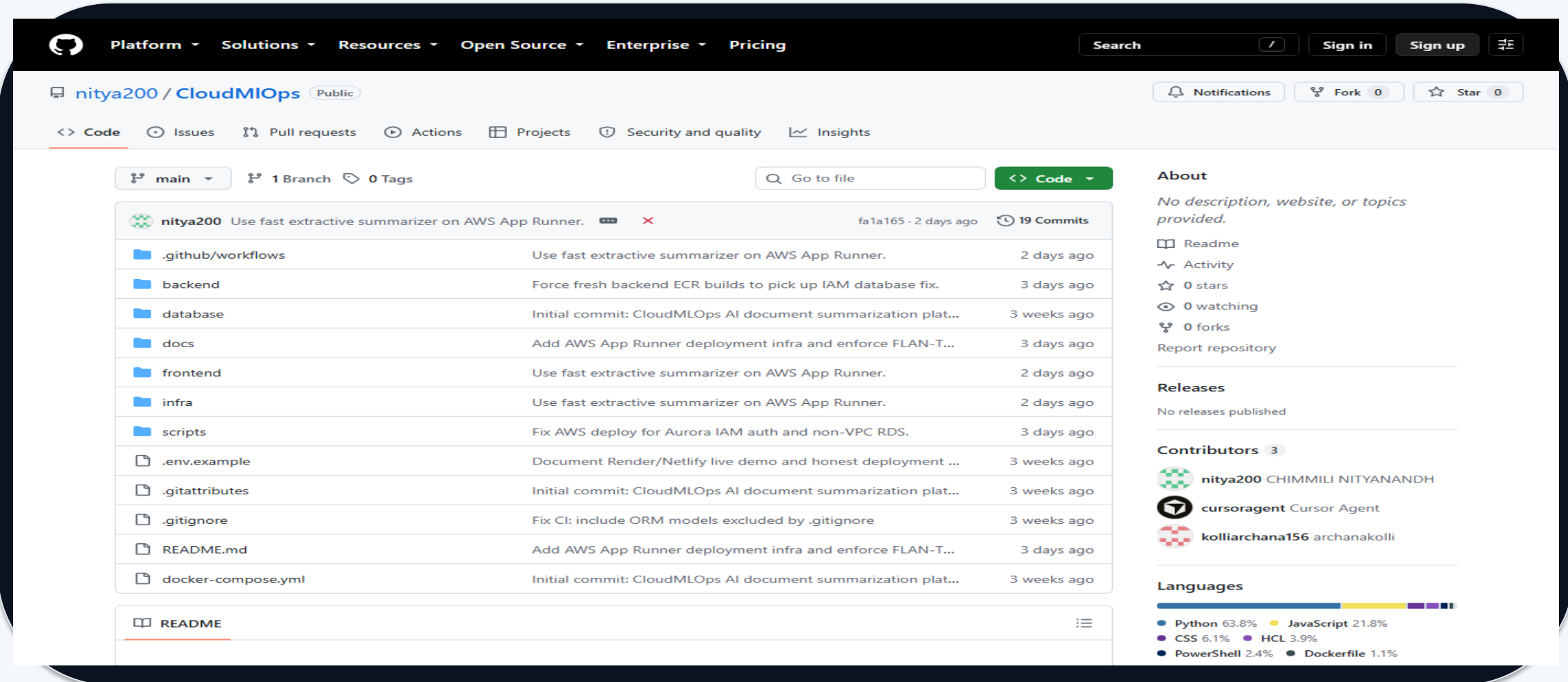
POST [/api/auth/login](#) Exchange credentials for a JWT access token

POST [/api/auth/logout](#) Revoke the current access token

Swagger UI served from the deployed FastAPI backend.

Source & CI/CD

GitHub repository



Platform Solutions Resources Open Source Enterprise Pricing

Search Sign in Sign up

nitya200 / CloudMLOps Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

main 1 Branch 0 Tags

Go to file

<> Code

nitya200 Use fast extractive summarizer on AWS App Runner. fa1a165 · 2 days ago 19 Commits

.github/workflows	Use fast extractive summarizer on AWS App Runner.	2 days ago
backend	Force fresh backend ECR builds to pick up IAM database fix.	3 days ago
database	Initial commit: CloudMLOps AI document summarization plat...	3 weeks ago
docs	Add AWS App Runner deployment infra and enforce FLAN-T...	3 days ago
frontend	Use fast extractive summarizer on AWS App Runner.	2 days ago
infra	Use fast extractive summarizer on AWS App Runner.	2 days ago
scripts	Fix AWS deploy for Aurora IAM auth and non-VPC RDS.	3 days ago
.env.example	Document Render/Netlify live demo and honest deployment ...	3 weeks ago
.gitattributes	Initial commit: CloudMLOps AI document summarization plat...	3 weeks ago
.gitignore	Fix CI: include ORM models excluded by .gitignore	3 weeks ago
README.md	Add AWS App Runner deployment infra and enforce FLAN-T...	3 days ago
docker-compose.yml	Initial commit: CloudMLOps AI document summarization plat...	3 weeks ago

README

About

No description, website, or topics provided.

Readme Activity 0 stars 0 watching 0 forks Report repository

Releases

No releases published

Contributors 3

- nitya200 CHIMMILI NITYANANDH
- cursoragent Cursor Agent
- kolliarchana156 archanakolli

Languages

Python 63.8% JavaScript 21.8% CSS 6.1% HCL 3.9% PowerShell 2.4% Dockerfile 1.1%

Repository with workflows for CI and container deployment.

Results & Outcomes

Verified deliverables

171 automated pytest cases across auth, documents, AI, admin, and migrations.

Production AWS deployment with connected PostgreSQL (see Live URLs slide).

Extractive summarizer on AWS for reliable responses within App Runner limits.

Terraform-managed infrastructure and GitHub Actions CI/CD pipeline.

Model registry with ROUGE evaluation, approve, and promote workflow.

Future Scope

Planned improvements (not yet implemented)

GPU hosting for FLAN-T5 abstractive summaries at scale.

CloudWatch alarms and SNS operational alerts.

Expanded benchmark evaluation where compute allows.

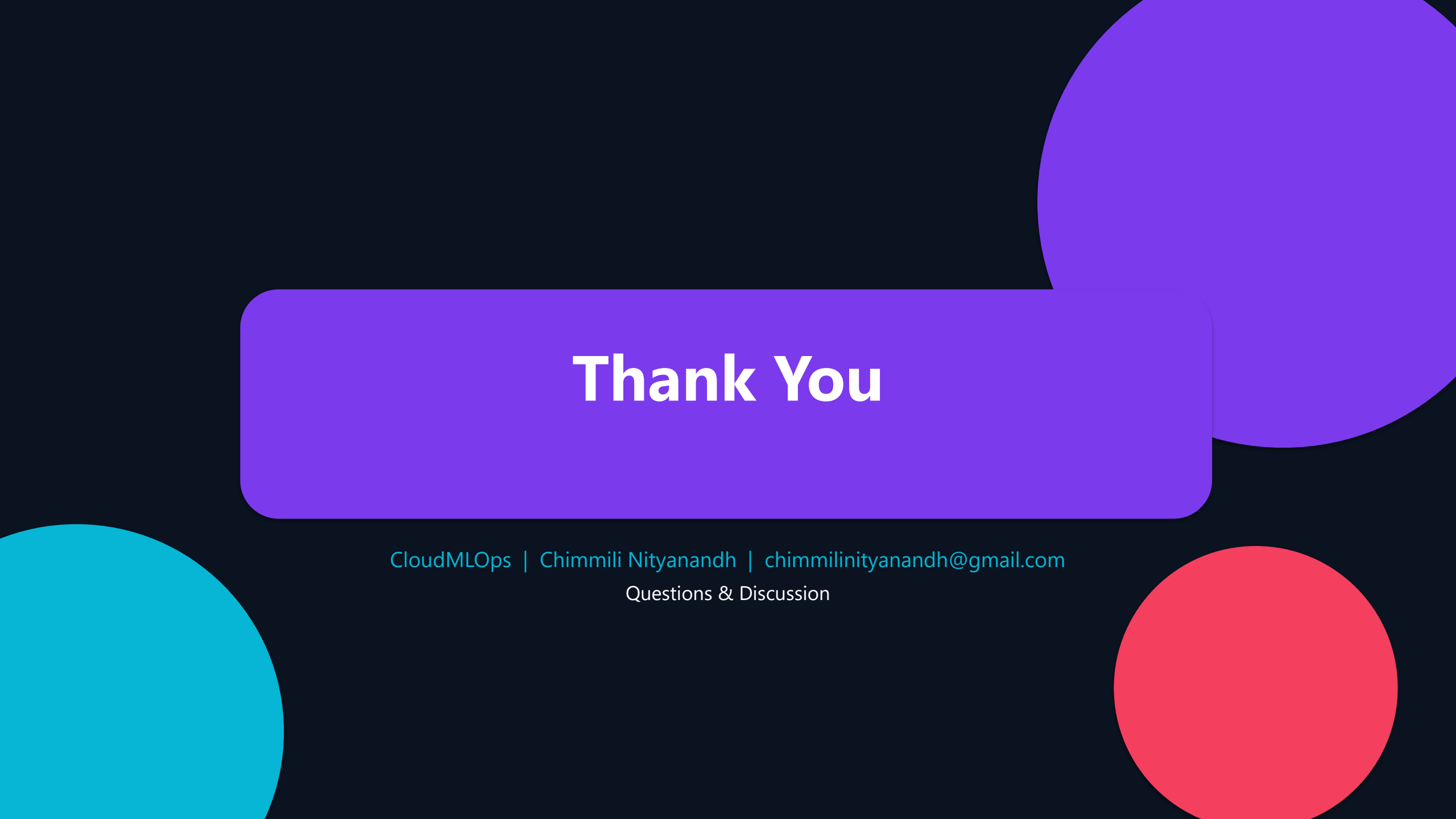
Conclusion

Summary

CloudMLOps delivers secure document summarization with PostgreSQL persistence.

Combines modern web stack, AI integration, and AWS MLOps practices.

Faculty-reviewable live URLs, automated tests, and admin analytics.



Thank You

CloudMLOps | Chimmili Nityanandh | chimmilinityanandh@gmail.com

Questions & Discussion